Introduction to Lambda Calculus

Lec8-3 CO523

BAKD

DoCE

27-01-2026

- A **formal mathematical system** for expressing computation
- Introduced by **Alonzo Church (1930s)**
- Foundation of **functional programming languages** (Lisp, ML, Haskell, Scheme)
- Focuses on **functions**, not machines or memory
- Everything is a **function** (even numbers, booleans, data structures)

- a formal mathematical system and universal model of computation that serves as the theoretical foundation for functional programming languages. It is considered the smallest universal programming language, capable of expressing any computable function using only function abstraction and application

Core Concepts of Lambda Calculus

The lambda calculus is a minimal language built on only three components:

- Variables: Placeholders for values, like x.
- Function Abstraction: The definition of an anonymous function with a parameter and a body, written as λx.M (a function that takes x and returns M).
- Application: The act of applying a function to an argument, written as M N (function M applied to argument N).

Computation in lambda calculus involves a process called beta-reduction, which is the formal rule for function application (substitution of the argument for the parameter in the function body).

Influence on Programming Languages

Lambda calculus has profoundly influenced the design and features of many programming languages:

- Functional Programming (FP) Languages:

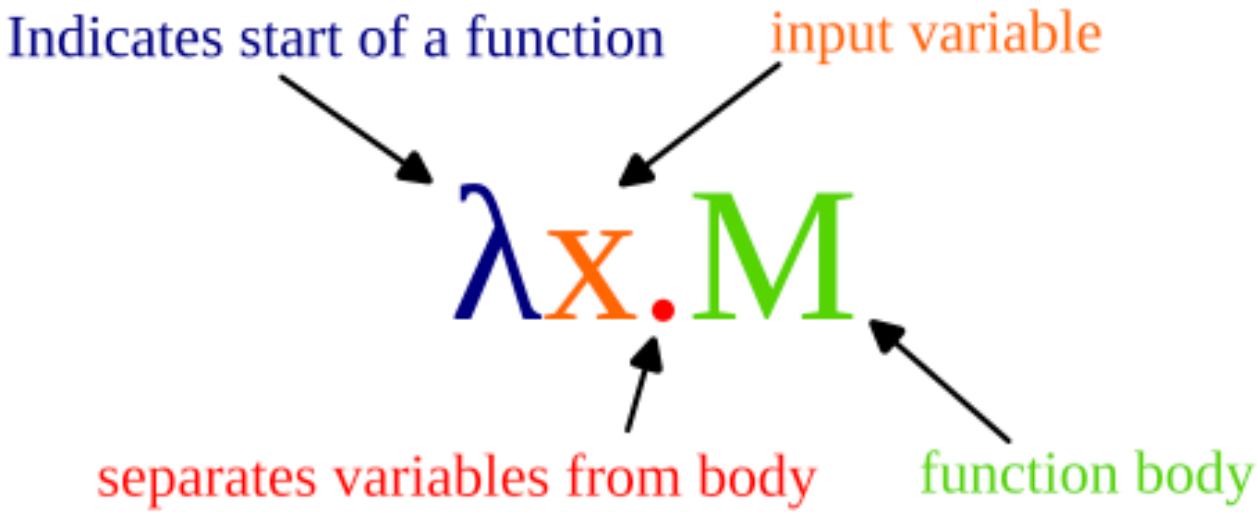
These languages (e.g., Haskell, Scheme, Clojure) are directly based on lambda calculus principles, treating functions as first-class citizens that can be passed as arguments, returned from other functions, and assigned to variables.

Key Features: Concepts derived from lambda calculus that appear in modern languages include:

- Anonymous functions/Lambda expressions: Small, unnamed functions (e.g., Python's lambda x: x + 1, JavaScript's (x) => x + 1).
- Higher-order functions: Functions that operate on other functions (e.g., map(), filter(), sorted() in Python).
- Closures: Nested functions that can access variables from their outer scope even after the outer function has finished executing.
- Immutability: The principle that data, once created, should not be changed, which simplifies reasoning about program state and facilitates concurrency.
- Type systems: Typed lambda calculi form the foundation for modern static type systems in languages like ML, Haskell, and Rust, helping ensure program correctness and safety.

- Why it Matters

While not used for practical programming in its raw form, lambda calculus is a powerful theoretical tool because it is Turing complete



The lambda abstraction decomposed. The λ indicates the start of a function. x is the input parameter. M is the body, separated by a dot separator "." from the input parameter.

https://youtu.be/ViPNHMSUcog

2. Why Lambda Calculus Matters in Programming Languages

- Theoretical basis of **functional programming**
- Helps understand:
  - Function application
  - Scope and binding
  - Recursion
  - Higher-order functions
- Influences:
  - Compilers and interpreters
  - Type systems
  - Language semantics

3. Basic Syntax

Lambda calculus has **three core constructs**:

1. **Variable**
   - Example: x, y, z
2. **Abstraction (Function Definition)**
   - Form: λx. M
   - Meaning: Function that takes x and returns M
   - Example: λx. x + 1
3. **Application (Function Call)**
   - Form: M N
   - Meaning: Apply function M to argument N
   - Example: (λx. x + 1) 5

4. Free and Bound Variables

- **Bound Variable:** Appears inside a lambda abstraction
  - Example: In λx. x + y, x is bound
- **Free Variable:** Not bound by any lambda
  - Example: In λx. x + y, y is free

5. Alpha, Beta, and Eta Conversions

5.1 Alpha Conversion (Renaming Variables)

- Changes bound variable names
- Example:
  - $\lambda x.\ x + 1 \equiv \lambda y.\ y + 1$
- Purpose: Avoid name conflicts

5.2 Beta Reduction (Function Application)

- Applies a function to an argument
- Rule: (λx. M) N → M[x := N]
- Example:
  - (λx. x + 1) 5 → 5 + 1 → 6

5.3 Eta Conversion (Simplification)

- Removes redundant abstractions
- Rule: λx. (f x) → f if x not free in f
- Example:
  - λx. (f x) ≡ f

6. Church Encoding (Data in Lambda Calculus)

6.1 Church Numerals

- Represent numbers as functions
- Example:
  - 0 ≡ λf. λx. x
  - 1 ≡ λf. λx. f x
  - 2 ≡ λf. λx. f (f x)
-

6.2 Boolean Values

- TRUE ≡ λt. λf. t
- FALSE ≡ λt. λf. f

6.3 Conditional Expression

- IF ≡ λb. λx. λy. b x y

7. Recursion and the Y-Combinator

- Lambda calculus has **no named functions**
- Recursion is achieved using **fixed-point combinators**
- Most famous: **Y-Combinator**

Definition:

- $Y = \lambda f.\ (\lambda x.\ f\ (x\ x))\ (\lambda x.\ f\ (x\ x))$

Use:

- Enables recursion like factorial, Fibonacci

8. Evaluation Strategies

- **Normal Order:**
  - Reduce outermost expressions first
  - Guarantees result if it exists
- **Applicative Order:**
  - Reduce arguments first
  - Used in most real languages (e.g., C, Java)

9. Typed vs Untyped Lambda Calculus

9.1 Untyped Lambda Calculus

- No data types
- Very powerful but unsafe
- Basis for theoretical models

9.2 Typed Lambda Calculus

- Every expression has a type
- Safer and more practical
- Basis for:
  - ML
  - Haskell
  - Type systems in modern languages

10. Relationship to Programming Languages

Lambda Calculus Concept	Programming Language Equivalent
Abstraction	Function definition
Application	Function call
Higher-order functions	Functions as parameters/returns
Church numerals	Integers
Y-combinator	Recursion

Example (Exam-Friendly)

- **Evaluate using beta reduction:**

- (λx. x * 2) (3 + 1)
- → (3 + 1) * 2
- → 4 * 2
- → 8

summery

- Lambda calculus is the **core theory** behind functional programming
- Everything is a **function**
- Computation = **function application + reduction**
- Foundation for:
  - Type systems
  - Language semantics
  - Compiler design